

Python
for DA

Summary Session 7



Преподаватель

Портрет

Имя Фамилия

Текущая должность

Количество лет опыта

Какой у Вас опыт - ключевые кейсы

Самые яркие проекты

Дополнительная информация по вашему усмотрению

Корпоративный e-mail

Социальные сети (по желанию)

Важно

-  Камера должна быть включена на протяжении всего занятия
-  В течение занятия вопросы задавать в чате или когда преподаватель спрашивает, есть ли у Вас вопросы
-  Вести себя уважительно и этично по отношению к остальным участникам занятия
-  Организационные вопросы по обучению решаются с кураторами, а не на тематических занятиях
-  Во время занятия будут интерактивные задания, будьте готовы включить камеру или демонстрацию экрана по просьбе преподавателя



ПОВТОРЕНИЕ ИЗУЧЕННОГО



Экспресс-опрос



Что такое Plotly?



Plotly

Это интерактивная библиотека для построения графиков с открытым исходным кодом, способна создавать сложные графики и интерактивные визуализации на языке Python. Построена на базе библиотеки Plotly JavaScript (plotly.js).

Ключевые особенности Plotly



Интерактивность



Настраиваемость



Интеграция



Опции экспорта

Сравнение Plotly с Matplotlib и Seaborn



Matplotlib

Интерактивность ограничена по сравнению с Plotly, а создание интерактивных графиков может потребовать больше настроек и кода.

Seaborn

Интерактивность не так развита, как в Plotly, и больше сосредоточена на улучшении визуальной эстетики и простоте создания сложных статистических графиков.

Установка библиотеки Plotly



```
pip install plotly
```

Установка ipywidgets



```
pip install ipywidgets
```

Создание первой визуализации

```
import plotly.express as px

data = {'x': [1, 2, 3, 4, 5], 'y': [4, 1, 8, 3, 10]}
fig = px.line(data, x='x', y='y', title='Простой линейный график')
fig.show()
```



Plotly Express

Это высокоуровневый интерфейс библиотеки Plotly, предназначенный для упрощения и повышения интуитивности визуализации данных.

Ключевые особенности Plotly Express



Простота



Универсальность



Интеграция



Интерактивность



Персонализация

Импорт модуля plotly.express



```
import plotly.express as px
```

Экспресс-опрос

?

Что такое линейные диаграммы?



Линейные диаграммы

Это фундаментальный инструмент визуализации данных, идеально подходящий для отображения тенденций во времени.



Простая линейная диаграмма

Отображает связь между двумя
переменными.

Пример построения линейных диаграмм



Набор данных

Создание линейной диаграммы

```
import plotly.express as px
import pandas as pd

df = pd.read_csv(
    'https://raw.githubusercontent.com/plotly/datasets/master/finance-charts-apple.csv'
)

fig = px.line(df, x='Date', y='AAPL.Close', title='AAPL.Close Trend Over Days')
fig.show()
```

Экспресс-опрос

?

Что такое `px.line()`?



px.line()

Это функция, которая используется для создания линейных графиков. Нужно указать кадр данных и столбцы для осей x и y.



Многолинейные диаграммы

Полезны для сравнения нескольких групп по одной оси x .

Многолинейные диаграммы

```
fig = px.line(df, x='Date', y='AAPL.Close', color='direction', title='Total AAPL.Close  
Trend by Direction')  
fig.show()
```

Настройка диаграмм

```
fig = px.line(df, x='Date', y='AAPL.Close', color='direction',  
              title='Customized Total AAPL.Close Trend by Direction',  
              markers=True, line_dash='direction')  
fig.show()
```

Экспресс-опрос



Что такое гистограммы?



Гистограммы

Это универсальный способ визуализации данных, включающих категориальные сравнения.



Базовая гистограмма

Отображает распределение числовой переменной по различным категориям.

Пример построения гистограммы



Набор данных "Gapminder"

Создание гистограммы

```
import plotly.express as px

df = px.data.gapminder().query("country=='Germany'")
df['popMillion'] = df['pop']/1000000

fig = px.bar(df, x='year', y='popMillion', title='Total Population of Germany by Year')
fig.show()
```

Экспресс-опрос

?

Что такое `px.bar()`?



`px.bar()`

Это функция, которая используется для создания столбчатых диаграмм.



Сложенные столбчатые диаграммы

Полезны для отображения распределения нескольких переменных, сложенных друг на друга, что дает представление об общем количестве и пропорциях подкатегорий.

Сложенные столбчатые диаграммы

```
fig = px.bar(df, x='year', y=['popMillion', 'lifeExp'],  
             title='Distribution of Measurements by Year', barmode='stack')  
fig.show()
```



Кластеризованные гистограммы

Размещают рядом друг с другом гистограммы для разных групп для удобства сравнения.

Кластеризованные гистограммы

```
fig = px.bar(df, x='year', y=['popMillion', 'lifeExp'],  
             title='Comparison of Measurements by Year', barmode='group')  
fig.show()
```

Настройка гистограмм

```
fig = px.bar(df, x='year', y='popMillion', color='year',  
             title='Population by Year with Color Coding')  
fig.show()
```



Круговые диаграммы

Это популярный метод отображения пропорционального распределения категориальных данных, где каждый кусочек “пирога” представляет собой вклад категории в общее количество.

Создание круговой диаграммы

```
import plotly.express as px

gapminder = px.data.gapminder()

gapminder_2007 = gapminder[gapminder['year'] == 2007]

fig = px.pie(gapminder_2007, names='continent', title='Country Distribution by
Continent in 2007')
fig.show()
```

Экспресс-опрос

?

Что такое rx.pie()?



px.pie()

Это функция, которая используется для создания круговых диаграмм.

Настройка круговых диаграмм

```
fig = px.pie(gapminder_2007, names='continent', color='continent',  
             title='Country Distribution by Continent in 2007',  
             hover_data=['country'], labels={'country':'Country Name'})  
fig.update_traces(textinfo='percent+label', pull=[0, 0.1, 0, 0, 0])  
fig.show()
```



Пончиковая диаграмма

Это разновидность круговой диаграммы, которая включает в себя отверстие в центре, часто используемое для отображения дополнительной информации.

Пончиковая диаграмма

```
fig = px.pie(gapminder_2007, names='continent', hole=0.3,  
             title='Country Distribution by Continent in 2007 (Donut Chart)')  
fig.show()
```



hole

Это функция, которая добавляет отверстие в центре круговой диаграммы, превращая ее в пончиковую диаграмму. Значение задает размер отверстия относительно радиуса круговой диаграммы.

Интерактивные функции

Масштабирование

Панорамирование

Экспресс-опрос

?

Что такое масштабирование?



Масштабирование

Это интерактивная функция, которая позволяет пользователям увеличить определенную часть графика, чтобы рассмотреть данные более детально.

Способы масштабирования

Масштаб бокса

Scroll Zoom

Нажмите и перетащите, чтобы
выбрать прямоугольную область
для увеличения

Способы масштабирования

Масштаб бокса

Scroll Zoom

Используйте колесо прокрутки
мыши для увеличения или
уменьшения масштаба

Экспресс-опрос



Что такое панорамирование?



Панорамирование

Это интерактивная функция, которая позволяет пользователям перемещать вид графика по горизонтали или вертикали, что особенно полезно после увеличения масштаба графика.



ВОПРОСЫ



Экспресс-опрос

?

Что такое диаграммы рассеяния?



Диаграммы рассеяния

Это фундаментальный инструмент визуализации данных, идеально подходящий для изучения взаимосвязей между двумя или более переменными.

Пример построения диаграмм рассеяния



Набор данных "Iris" из Plotly Express

Создание базовой диаграммы рассеяния

```
import plotly.express as px
```

```
iris = px.data.iris()
```

```
fig = px.scatter(iris, x='sepal_length', y='sepal_width', color='species', title='Iris  
Dataset: Sepal Width vs Sepal Length')
```

```
fig.show()
```



Пузырьковые диаграммы

Это расширение диаграмм рассеивания, где размер каждого маркера (пузырька) может представлять дополнительное измерение данных.

Создание пузырьковой диаграммы

```
fig = px.scatter(iris, x='sepal_length', y='sepal_width', color='species',  
                  size='petal_length',  
                  title='Iris Dataset: Bubble Chart of Sepal Dimensions with Petal  
Length as Size')  
fig.show()
```

Установка модуля statsmodels

```
fig = px.scatter(iris, x='sepal_length', y='sepal_width', color='species',  
                 trendline='ols') # Ordinary Least Squares regression line  
fig.show()
```

Экспресс-опрос

?

Что такое Box plot?



Box plot

Это стандартный способ отображения распределения данных на основе сводки из пяти чисел: минимум, первый квартиль (Q1), медиана, третий квартиль (Q3) и максимум.

Пример построения Box plot



Набор данных "Tips" из Plotly Express

Базовая настройка боксплотта

```
tips = px.data.tips()

fig = px.box(tips, y='total_bill', title='Distribution of Total Bills')
fig.show()
```

Экспресс-опрос

?

Что такое `px.box()`?



`px.box()`

Это функция, которая используется для создания боксплотов.

Сравнение распределения общих счетов по дням

```
fig = px.box(tips, x='day', y='total_bill', title='Distribution of Total Bills')  
fig.show()
```

Настройка Box plot

```
fig = px.box(tips, x='day', y='total_bill', color='smoker',
             notched=True, # Добавляет засечки для обозначения доверительного
                           интервала вокруг медианы
             points='all', # Отображает все точки на боксплоте
             title='Customized Distribution of Total Bills by Day and Smoker
Status')
fig.show()
```

Горизонтальное отображение Box plot

```
fig = px.box(tips, x='total_bill', y='day', orientation='h', # Устанавливает
горизонтальную ориентацию
               title='Horizontal Distribution of Total Bills by Day')
fig.show()
```

Экспресс-опрос

?

Зачем нужны тепловые карты?



Тепловые карты

Это графическое представление данных, в котором отдельные значения, содержащиеся в матрице, представлены в виде цветов.

Пример построения тепловой карты



Набор данных "Рейсы" из seaborn

Создание базовой тепловой карты

```
import plotly.express as px
import seaborn as sns

flights = sns.load_dataset("flights")

fig = px.imshow(flights.pivot_table(index='month', columns='year',
                                     values='passengers'),
                labels=dict(x="Year", y="Month", color="Passengers"),
                title='Monthly Passenger Numbers Over Years')

fig.show()
```

Экспресс-опрос

?

Что такое `px.imshow()`?



`px.imshow()`

Это функция, которая используется для создания графиков изображений, в том числе тепловых карт.

Настройка тепловой карты

```
fig = px.imshow(flights.pivot_table(index='month', columns='year',
values='passengers'),
                labels=dict(x="Year", y="Month", color="Passengers"),
                title='Customized Monthly Passenger Numbers Over Years',
                aspect="auto", # Настраивает соотношение сторон
                color_continuous_scale='Viridis', # Изменяет цветовую шкалу
                text_auto=True) # Добавляет аннотации

fig.show()
```

Экспресс-опрос

?

Что такое лучевые диаграммы?



Лучевые диаграммы

Это диаграммы, которые эффективны для визуализации иерархических структур данных и пропорций внутри них, показывая, как одно кольцо, уровень в иерархии, разбивается на составляющие его части.

Пример построения лучевых диаграмм



Набор данных "Tips" из Plotly Express

Создание лучевой диаграммы

```
import plotly.express as px

tips = px.data.tips()

fig = px.sunburst(tips, path=['day', 'time', 'sex'], values='total_bill',
                  color='time', title='Distribution of Total Bills by Day, Time,
and Sex')
fig.show()
```

Экспресс-опрос

?

Что такое px.sunburst()?



`px.sunburst()`

Это функция, которая используется для создания диаграмм в виде солнечных лучей.

Настройка пончиковой диаграммы

```
fig = px.sunburst(tips, path=['day', 'time', 'sex'], values='total_bill',  
                  color='total_bill', # Теперь раскраска по общей сумме счета  
                  color_continuous_scale='Blues', # Изменяет цветовую шкалу  
                  title='Customized Distribution of Total Bills by Day, Time,  
and Sex')  
fig.update_traces(textinfo='label+percent entry') # Показывает метки и процент  
на каждом уровне  
  
fig.show()
```

Пример настройки графиков



Набор данных "Iris" из Plotly Express

Создание базовой диаграммы рассеяния

```
import plotly.express as px

iris = px.data.iris()

fig = px.scatter(iris, x='sepal_length', y='sepal_width', color='species',
title='Iris Dataset: Sepal Width vs Sepal Length')
fig.show()
```

Добавление заголовков и меток осей

```
fig.update_layout(  
    title='Iris Flower Measurements',  
    xaxis_title='Sepal Length (cm)',  
    yaxis_title='Sepal Width (cm)'  
)  
fig.show()
```



title

Это функция, которая устанавливает основной заголовок графика.



`xaxis_title` и `yaxis_title`

Это функции, которые устанавливают заголовки для оси x и оси y соответственно.

Добавление легенд

```
fig.update_layout(  
    legend_title='Species',  
    legend=dict(  
        x=1,  # Положение легенды по горизонтали (1 - крайний справа)  
        y=1,  # Положение легенды по вертикали (1 - верхняя)  
        bordercolor='Black',  
        borderwidth=1  
    )  
)  
fig.show()
```



legend_title

Это функция, которая устанавливает заголовок легенды.



legend (легенда)

Это словарь, позволяющий настраивать положение и внешний вид легенды. `x` и `y` управляют положением, а `bordercolor` и `borderwidth` настраивают границу.

Добавление аннотаций

```
fig.add_annotation(  
    x=5, y=3.5,  
    text="Outlier",  
    showarrow=True,  
    arrowhead=1,  
    ax=20, # Положение хвоста стрелки (сдвиг по оси x)  
    ay=-30 # Положение хвоста стрелки (сдвиг по оси y)  
)  
fig.show()
```

Настройка внешнего вида графика

```
fig.update_layout(  
    plot_bgcolor='lightgrey',  # Цвет фона области построения  
    paper_bgcolor='white',     # Цвет фона за пределами области построения  
    font=dict(  
        family="Courier New, monospace",  
        size=12,  
        color="RebeccaPurple"  
    )  
)  
fig.show()
```



`plot_bgcolor` и `paper_bgcolor`

Это функции, которые устанавливают цвета фона графика и области вокруг графика соответственно.



font (шрифт)

Это словарь для настройки стиля, размера и цвета шрифта, используемого в графике.



ВОПРОСЫ





3D диаграммы рассеяния и графики поверхности

Это инструменты для визуализации сложных наборов данных с трехмерными данными, которые позволяют исследовать взаимосвязи между тремя переменными, отображая их в трехмерном пространстве.

Экспресс-опрос

?

Что такое `px.scatter_3d()`?



`px.scatter_3d()`

Это функция, которая создает трехмерную диаграмму рассеяния. Она требует указания фрейма данных и столбцов, которые будут использоваться для осей x, y и z.

Создание 3D диаграммы рассеяния

```
import plotly.express as px

df = px.data.iris()

fig = px.scatter_3d(df, x='sepal_length', y='sepal_width', z='petal_width',
color='species')
fig.show()
```

Создание графика поверхности



Чтобы создать график поверхности, нам нужна структурированная сетка точек (x, y) и соответствующие значения z , которые определяют высоту поверхности в каждой точке



go.Surface()

Это функция, которая используется для создания графиков поверхности. Она требует матрицу значений z и, опционально, матрицы x и y , если сетка не 1:1.

Создание графика поверхности

```
import plotly.graph_objects as go
import numpy as np

x = np.linspace(-5, 5, 50)
y = np.linspace(-5, 5, 50)
x, y = np.meshgrid(x, y)
z = np.sin(np.sqrt(x ** 2 + y ** 2))

fig = go.Figure(data=[go.Surface(z=z, x=x, y=y)])
fig.update_layout(title='3D Surface Plot', autosize=False,
                    width=500, height=500,
                    margin=dict(l=65, r=50, b=65, t=90))

fig.show()
```

Экспресс-опрос

?

Что такое `update_layout()`?



`update_layout()`

Это метод, который настраивает макет фигуры, включая заголовок, размер и поля.



Graph Objects (GO)

Это модуль в Plotly, который обычно импортируется как `plotly.graph_objects`, представляет собой мощный низкоуровневый интерфейс, который обеспечивает детальный контроль над созданием и настройкой фигур.

Импорт модуля Graph Objects



```
import plotly.graph_objects as go
```

Экспресс-опрос

?

Каковы основные компоненты фигуры Plotly?

Основные компоненты фигуры Plotly

Данные

Фактические точки
данных, которые нужно
построить

Макет

Эстетические аспекты
фигуры

Рамки

Каждый кадр анимации



`go.Figure()`

Это команда, которая инициализирует новый объект фигуры.



`add_trace()`

Это команда, которая добавляет новую трассировку к фигуре.



`update_layout()`

Это команда, которая обновляет настройки макета фигуры.



`fig.show()`

Это команда, которая отображает фигуру в интерактивном окне.

Создание простой линейной диаграммы

```
fig = go.Figure()

fig.add_trace(go.Scatter(x=[1, 2, 3, 4], y=[10, 11, 12, 13], mode='lines'))

fig.update_layout(title='Simple Line Chart', xaxis_title='X Axis',
yaxis_title='Y Axis')

fig.show()
```



`update_traces()`

Это команда, которая изменяет свойства трассировок.



`add_annotation()`

Это команда, которая добавляет аннотации к графику.



`update_layout()`

Это команда, которая выполняет дальнейшую настройку макета.

Настройка внешнего вида графика

```
fig.update_traces(line=dict(color='RoyalBlue', width=4))  
fig.add_annotation(x=3, y=12, text="Highlight", showarrow=True, arrowhead=1)  
fig.update_layout(plot_bgcolor='lightgrey')
```



`plotly.subplots.make_subplots`

Это функция, которая позволяет создавать сложные макеты с общими осями x, y или обеими.

Экспресс-опрос

?

Что такое `make_subplots`?



`make_subplots`

Это функция, которая используется, чтобы создать фигуру с несколькими подграфиками. Она инициализирует макет подграфиков и возвращает объект `figure`, который можно заполнить данными.

Использование функции make_subplots

```
from plotly.subplots import make_subplots
import plotly.graph_objects as go

fig = make_subplots(rows=2, cols=1)

fig.add_trace(go.Scatter(x=[1, 2, 3], y=[4, 5, 6]), row=1, col=1)
fig.add_trace(go.Scatter(x=[1, 2, 3], y=[6, 7, 8]), row=2, col=1)

fig.show()
```



`shared_xaxes` и `shared_yaxes`

Это параметры, которые позволяют настроить общие оси.

Пример с общими осями Y

```
fig = make_subplots(rows=2, cols=2, shared_yaxes=True)

fig.add_trace(go.Scatter(x=[1, 2, 3], y=[2, 3, 4]), row=1, col=1)
fig.add_trace(go.Scatter(x=[20, 30, 40], y=[5, 5, 5]), row=1, col=2)
fig.add_trace(go.Scatter(x=[2, 3, 4], y=[600, 700, 800]), row=2, col=1)
fig.add_trace(go.Scatter(x=[4000, 5000, 6000], y=[7000, 8000, 9000]), row=2, col=2)

fig.update_layout(height=600, width=600, title_text="Multiple Subplots with Shared Y-Axes")
fig.show()
```



`vertical_spacing` и `horizontal_spacing`

Это параметры, которые позволяют настроить расстояние между подграфиками.



`subplot_titles`

Это параметр, который позволяет добавить заголовки к каждому подграфику.

Пример с настраиваемым макетом

```
fig = make_subplots(
    rows=2, cols=2,
    shared_yaxes=True,
    vertical_spacing=0.1,
    subplot_titles=("Plot 1", "Plot 2", "Plot 3", "Plot 4")
)

fig.add_trace(go.Scatter(x=[1, 2, 3], y=[2, 3, 4]), row=1, col=1)
fig.add_trace(go.Scatter(x=[20, 30, 40], y=[5, 5, 5]), row=1, col=2)
fig.add_trace(go.Scatter(x=[2, 3, 4], y=[600, 700, 800]), row=2, col=1)
fig.add_trace(go.Scatter(x=[4000, 5000, 6000], y=[7000, 8000, 9000]), row=2,
col=2)

fig.update_layout(height=600, width=600, title_text="Customized Subplots
Layout")
fig.show()
```

Экспресс-опрос

?

Какие интерактивные функции в Plotly вы помните?

Интерактивные функции в Plotly



Масштабирование



Панорамирование



Наведение курсора



Пользовательские кнопки и ползунки



Масштабирование и панорамирование

Это основные интерактивные функции, которые позволяют пользователям сосредоточиться на определенных областях графика или более детально изучить его данные.

Масштабирование и панорамирование

```
import plotly.graph_objects as go

fig = go.Figure(data=go.Scatter(x=[1, 2, 3], y=[4, 5, 6]))
fig.update_layout(
    title='Zoom and Pan Example',
    xaxis=dict(rangeslider=dict(visible=True)),
    yaxis=dict(fixedrange=False)
)
fig.show()
```



rangeslider

Это команда, которая добавляет ползунок диапазона к оси x, позволяя пользователям выбирать диапазон для увеличения.



Всплывающие подсказки

Это функция, которая предоставляет дополнительную информацию о точках данных, когда пользователь наводит на них курсор.

Всплывающие подсказки

```
fig = go.Figure(data=go.Scatter(x=[1, 2, 3], y=[4, 5, 6], mode='markers',  
                                hoverinfo='x+y+text', text=['Point 1', 'Point  
2', 'Point 3'])))  
fig.show()
```



hoverinfo

Это команда, которая указывает информацию, которая должна отображаться при наведении курсора на точки.



Добавление кнопок и выпадающих меню

Это функции, которые используются для динамического изменения визуализации.

Добавление кнопок и выпадающих меню

```
fig = go.Figure(data=[go.Scatter(x=[1, 2, 3], y=[4, 5, 6], mode='markers')])

fig.update_layout(
    updatemenus=[
        dict(
            buttons=list([
                dict(args=['type', 'scatter'], label='Scatter Plot',
method='restyle'),
                dict(args=['type', 'bar'], label='Bar Chart', method='restyle')
            ]),
            direction='down',
            showactive=True,
        )
    ]
)
fig.show()
```



updatemenus

Это команда, которая настраивает
выпадающие меню или кнопки на графике.



Параметры конфигурации

Это функции, которые позволяют настраивать параметры взаимодействия с помощью параметров конфигурации.

Параметры конфигурации

```
fig.show(config={  
    'displayModeBar': True, # Всегда показывать панель режимов  
    'scrollZoom': True # Включить масштабирование прокрутки  
})
```



displayModeBar

Это команда, которая управляет видимостью панели режимов, которая содержит инструменты для взаимодействия с графиком.



displayModeBar

Это команда, которая позволяет пользователям увеличивать и уменьшать масштаб с помощью колеса прокрутки на мыши.



Анимация в Plotly

Это способ визуализации изменений во времени или переходов между различными состояниями данных. Они помогают выявить закономерности, показать эволюцию данных и сделать сложные данные более понятными.



`px.scatter()`

Это команда, которая создает диаграмму разброса.

Plotly Express

```
import plotly.express as px

df = px.data.gapminder()

fig = px.scatter(df, x="gdpPercap", y="lifeExp", animation_frame="year",
                animation_group="country",
                size="pop", color="continent", hover_name="country",
                log_x=True, size_max=55, range_x=[100,100000], range_y=[25,90])

fig.show()
```



ВОПРОСЫ



Экспресс-опрос



Что такое Dash?



Dash

Это фреймворк, разработанный компанией Plotly для создания интерактивных веб-приложений для работы с данными с помощью Python.

Технологии обеспечения Dash



Функции и возможности



Интерактивность



Простота использования



Настраиваемость



Dash Enterprise

Это расширенная платформа,
предоставляющая дополнительные
возможности для развертывания,
масштабирования и интеграции

Установка основного пакета Dash



```
pip install dash
```

Рендерер Dash



```
pip install dash-renderer
```



dash-renderer

Это компонент Dash, который отображает компоненты приложения в веб-браузере.

HTML-компоненты Dash



```
pip install dash-html-components
```

Основные компоненты Dash



```
pip install dash-core-components
```

Запрос установленных пакетов



```
pip show dash
```

Экспресс-опрос



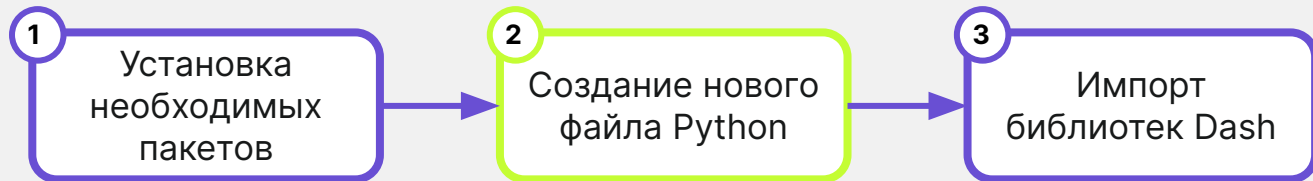
Что такое VS Code?



Visual Studio Code (VS Code)

Это популярный редактор кода,
поддерживающий широкий спектр языков
программирования, включая Python.

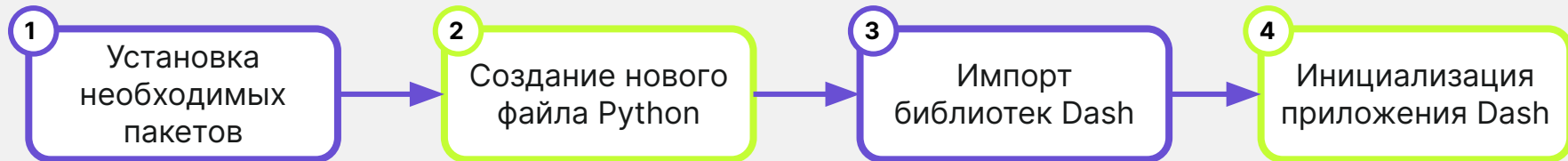
Настройка базовой среды Dash в VS Code



Импорт библиотек Dash

```
from dash import Dash, html, dcc  
import plotly.express as px
```

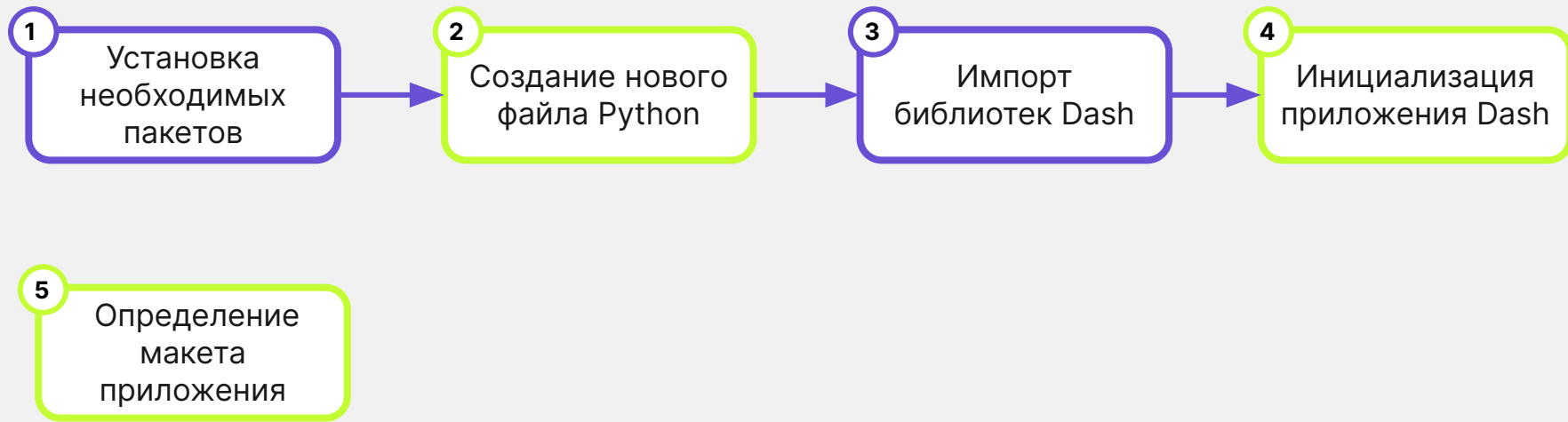
Настройка базовой среды Dash в VS Code



Инициализация приложения Dash

```
app = Dash(__name__)
```

Настройка базовой среды Dash в VS Code



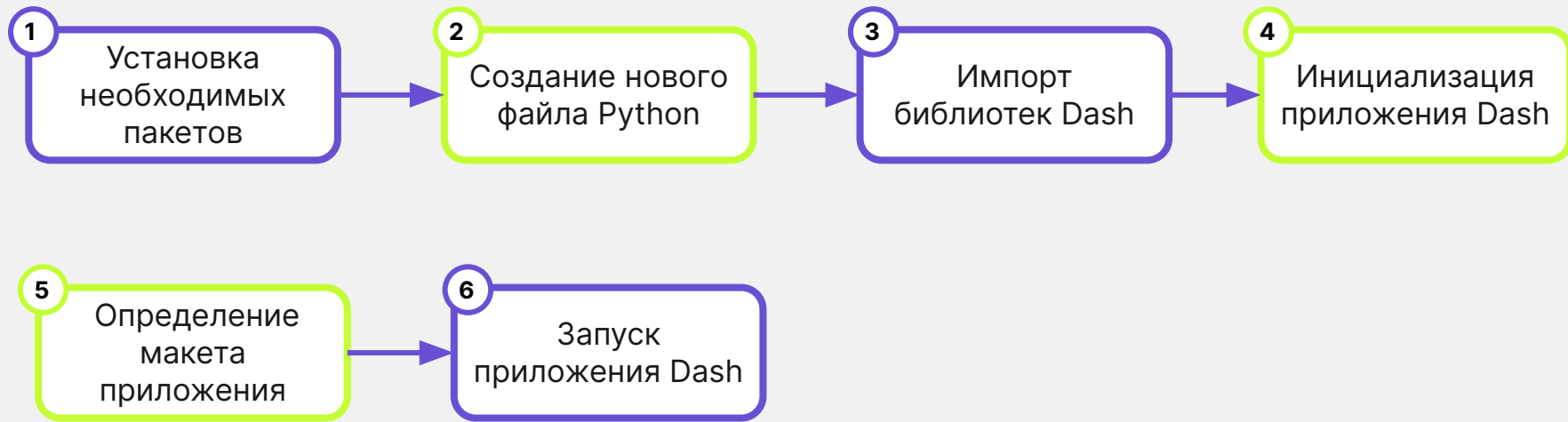
Определение макета приложения

```
df = px.data.iris()

fig = px.scatter(df, x='sepal_width', y='sepal_length', color='species')

app.layout = html.Div([
    html.H1("Визуализация набора данных Iris"),
    dcc.Graph(figure=fig)
])
```

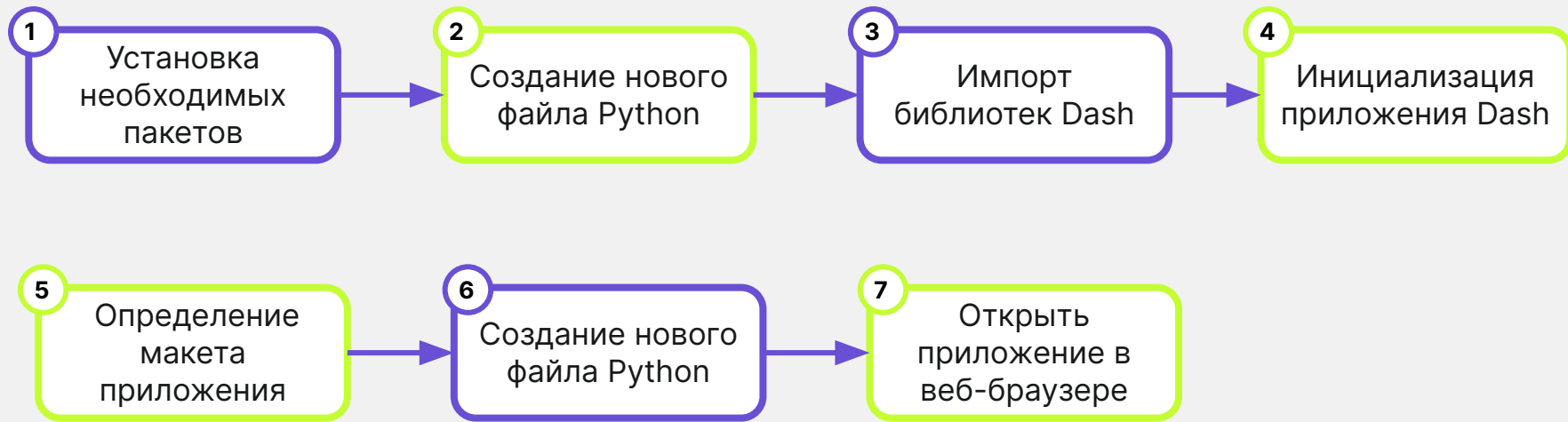
Настройка базовой среды Dash в VS Code



Запуск приложения Dash

```
if __name__ == '__main__':  
    app.run_server(debug=True)
```

Настройка базовой среды Dash в VS Code



Экспресс-опрос



Что такое HTML-контур?



HTML-контур

Это контур, который обычно включает в себя несколько ключевых разделов: верхний колонтитул, боковая панель, область основного содержимого и нижний колонтитул.



Заголовок

Содержит заголовок панели, а также навигацию верхнего уровня или важную информацию, которую пользователи должны видеть немедленно.

Заголовок

```
from dash import html
```

```
header = html.Header(html.H1('My Dashboard', className='header-title'))
```



Боковая панель

Полезна для размещения навигационных ссылок, фильтров или других интерактивных элементов управления, которые влияют на данные, отображаемые в основной области содержимого.

Боковая панель

```
sidebar = html.Div([
    html.H2('Navigation', className='sidebar-title'),
    html.Ul([
        html.Li(html.A('Главная', href='/')),
        html.Li(html.A('Страница 1', href='/page-1')),
        html.Li(html.A('Страница 2', href='/page-2'))
    ])
], className='sidebar')
```



Область основного контента

Это центральная часть приборной панели, где будет отображаться основная информация, визуализации данных или контент.

Область основного контента

```
main_content = html.Section([
    html.Article([
        html.H3('Заголовок основного контента'),
        html.P('Некоторые важные материалы или визуализации данных находятся здесь.')
    ], className='main-article')
], className='main-content')
```



Нижний колонтитул

Содержит информацию об авторских правах, контактные данные или ссылки на соответствующие ресурсы.

Нижний колонтитул

```
footer = html.Footer([  
    html.P('Copyright (c) 2024 My Dashboard'),  
    html.P('Больше информации здесь')  
, className='footer')
```

Объединение разделов в макет приложения Dash

```
from dash import Dash, html

app = Dash(__name__)
app.layout = html.Div([
    header,
    sidebar,
    main_content,
    footer
], className='dashboard-container')

if __name__ == '__main__':
    app.run_server(debug=True)
```



`dash_core_components (dcc)`

Это библиотека в составе фреймворка Dash, которая предоставляет компоненты более высокого уровня, чем HTML-элементы, разработанные специально для интерактивности и визуализации данных.

Экспресс-опрос



Что такое Graph?



Graph

Это инструмент в `dash_core_components`, который используется для встраивания диаграмм Plotly в приложения Dash и придания им интерактивности.

Компонент Graph

```
from dash import dcc

# Пример встраивания графика Plotly в Dash
graph_component = dcc.Graph(
    id='example-graph',
    figure={
        'data': [
            {'x': [1, 2, 3], 'y': [4, 1, 2], 'type':'bar', 'name':'SF'}
            {'x': [1, 2, 3], 'y': [2, 4, 5], 'type':'bar', 'name':'Montréal'}
        ],
        'layout': {
            'title': 'Dash Data Visualization'
        }
    }
)
```



dcc.Graph

Это функция, которая используется для отображения интерактивных визуализаций данных.

Экспресс-опрос



Что такое Dropdown?



Dropdown

Это компонент, который позволяет пользователям выбирать из списка опций. Используется для управления отображением данных в других компонентах, например в графиках.

Компонент Dropdown

```
dropdown_component = dcc.Dropdown(
    id='my-dropdown',
    options=[
        {'label': 'San Francisco', 'value': 'SF'},
        {'label': 'Montreal', 'value': 'MTL'}
    ],
    value='SF'
)
```



dcc.DropDown

Это функция, которая создает выпадающее меню с возможностью выбора опций.

Экспресс-опрос



Что такое Slider?



Slider

Это компонент, который предоставляет пользователям возможность выбрать числовое значение из диапазона.

Компонент Slider

```
slider_component = dcc.Slider(  
    min=0,  
    max=20,  
    step=0.5,  
    value=10,  
    marks={i: str(i) for i in range(21)}  
)
```



dcc.Slider

Это функция, которая создает входной ползунок



Обратные вызовы

Прослушивают изменения свойств компонентов и соответствующим образом обновляют другие компоненты.

Обратные вызовы

```
from dash import Input, Output, callback

@callback(
    Output('example-graph', 'figure'),
    Input('my-dropdown', 'value')
)
def update_graph(selected_value):
    # Логика обновления графика на основе выбранного значения
    pass
```



@callback

Это декоратор для определения функции обратного вызова, которая обновляет части приложения в ответ на пользовательский ввод.

Пример приложения

```

from dash import Dash, html, dcc, callback, Output, Input
import plotly.express as px

df = px.data.iris()

app = Dash()

app.layout = [
    html.Div(children='My First App with Graph and Controls'),
    dcc.RadioItems(options=['sepal_width', 'petal_length', 'petal_width'], value='sepal_width',
id='controls-and-radio-item'),
    dcc.Graph(figure={}, id='controls-and-graph')
]

@callback(
    Output(component_id='controls-and-graph', component_property='figure'),
    Input(component_id='controls-and-radio-item', component_property='value')
)
def update_graph(col_chosen):
    fig = px.scatter(df, x=col_chosen, y='sepal_length', color='species')
    return fig

if __name__ == '__main__':
    app.run_server(debug=True)

```

Домашнее задание

Создайте простое приложение Dash, которое позволяет пользователям выбрать континент из выпадающего меню и динамически отображает гистограмму ожидаемой продолжительности жизни для выбранного континента, используя набор данных Gapminder.

1. Инициализируйте приложение Dash.
2. Определите макет, включающий `dcc.Dropdown` для выбора континента и `dcc.Graph` для отображения гистограммы.
3. Наполните выпадающие опции континентами из набора данных Gapminder.
4. Напишите функцию обратного вызова, которая обновляет гистограмму в зависимости от выбранного континента.
5. Гистограмма должна отображать распределение `lifeExp` для выбранного континента.
6. Добавьте к гистограмме соответствующие заголовки и метки осей.



ВОПРОСЫ





ЗАДАНИЕ



Задание

Завершить работу над задачами с практикумов 13 и 14.





Задание 1

Создайте диаграмму рассеяния для визуализации взаимосвязи между двумя переменными. Используйте набор данных `tips`, доступный в Plotly Express. Постройте график `total_bill` на оси `x` и `tip` на оси `y`. Раскрасьте точки по времени (обед или ужин).

Задание 2

Создайте диаграмму коробок для анализа распределения переменной. Используйте набор данных `tips`. Создайте бокс-диаграмму для `total_bill`. Разделите данные по дням, чтобы увидеть распределение для каждого дня.

Задание 3



Создайте гистограмму, чтобы понять распределение частот переменной. Используйте набор данных `tips`.
Создайте гистограмму для `total_bill`. Установите число бинов равным 20.

Задание 4



Создайте линейный график для наблюдения тенденций за определенный период. Используйте набор данных `garminder`, доступный в Plotly Express. Отметьте год на оси `x` и `lifeExp` на оси `y`. Отфильтруйте данные, чтобы включить в них только Германию.

Задание 5



Создайте круговую диаграмму для анализа соотношения категорий. Используйте набор данных tips. Создайте круговую диаграмму, показывающую долю общих счетов по дням.

Задание 6



Создайте тепловую карту для визуализации корреляции между несколькими переменными. Используйте набор данных iris, доступный в Plotly Express. Рассчитайте корреляционную матрицу и визуализируйте ее с помощью тепловой карты.

Задание 1

Создайте 3D диаграмму рассеяния на основе набора данных Iris для визуализации взаимосвязи между длиной чашелистика, шириной чашелистика и длиной лепестка с разбивкой по видам.

1. Загрузите набор данных Iris с помощью `plotly.express.data.iris()`.
2. Используйте `plotly.express.scatter_3d` для создания трехмерной диаграммы рассеяния.
3. В качестве оси x задайте длину чашелистика (`sepal_length`), оси y - ширину чашелистика (`sepal_width`), а оси z - длину лепестка (`petal_length`).
4. Раскрасьте точки в соответствии с колонкой видов.
5. Добавьте соответствующие метки осей и заголовок к графику.

Задание 2

Создайте анимированную диаграмму рассеяния, показывающую динамику ВВП на душу населения по годам для стран из набора данных Gapminder, сгруппированных по континентам.

1. Загрузите набор данных Gapminder с помощью `plotly.express.data.gapminder()`.
2. Используйте `plotly.express.scatter` для создания диаграммы рассеяния.
3. Установите `gdpPerCap` в качестве оси x, а `lifeExp` - в качестве оси y.
4. Используйте параметр `animation_frame`, чтобы анимировать график по столбцу `year`.
5. Раскрасьте точки в соответствии с колонкой континентов.
6. Убедитесь, что у графика есть осмысленный заголовок, метки осей и ползунок анимации правильно обозначен.

Задание 3



Используя набор данных Iris, создайте макет подграфика с 1 строкой и 2 столбцами: первый график - диаграмма рассеяния длины чашелистика по сравнению с длиной лепестка, а второй - гистограмма, показывающая среднюю ширину чашелистика для каждого вида.

1. Загрузите набор данных Iris с помощью `plotly.express.data.iris()`.
2. Используйте `plotly.subplots.make_subplots`, чтобы определить макет подграфика с 1 строкой и 2 столбцами.
3. Для первого подграфика используйте `plotly.graph_objects.Scatter`, чтобы создать диаграмму разброса длины чашелистика и длины лепестка.
4. Для второго подграфика рассчитайте среднюю ширину чашелистика для каждого вида и с помощью `plotly.graph_objects.Bar` создайте гистограмму.
5. Убедитесь, что оба графика имеют одинаковый масштаб оси y.
6. Добавьте соответствующие заголовки, метки осей и легенду.



ВОПРОСЫ





РАЗБОР ДОМАШНЕГО ЗАДАНИЯ



Домашнее задание

1. Создать диаграмму рассеивания с дополнительной сложностью для визуализации взаимосвязи между двумя переменными. Используйте набор данных `tips`, доступный в Plotly Express. Постройте график `total_bill` на оси `x` и `tip` на оси `y`. Раскрасьте точки по времени (обед или ужин). Добавьте на график линию тренда и настройте размер маркеров в зависимости от размера вечеринки.
2. Создать гистограмму с расширенными возможностями настройки. Используйте набор данных `tips`. Создайте гистограмму для `total_bill`. Установите число бинов на 20 и цвет по времени.

Домашнее задание

3. Создайте линейный график с несколькими линиями для сравнения тенденций. Используйте набор данных `gapminder`, доступный в Plotly Express. Отметьте год на оси `x` и `gdpPerCap` на оси `y`. Отфильтруйте данные, чтобы включить в них только страны "Канада" и "Германия". Используйте разные стили линий для каждой страны.

4. Создать тепловую карту для визуализации корреляции между несколькими переменными с помощью интерактивных функций. Используйте набор данных `iris`, доступный в Plotly Express. Рассчитайте корреляционную матрицу и визуализируйте ее с помощью тепловой карты. Добавьте наведенную информацию, показывающую значения корреляции.

Домашнее задание

Создайте простое приложение Dash, которое позволяет пользователям выбрать континент из выпадающего меню и динамически отображает гистограмму ожидаемой продолжительности жизни для выбранного континента, используя набор данных Gapminder.

1. Инициализируйте приложение Dash.
2. Определите макет, включающий `dcc.Dropdown` для выбора континента и `dcc.Graph` для отображения гистограммы.
3. Наполните выпадающие опции континентами из набора данных Gapminder.
4. Напишите функцию обратного вызова, которая обновляет гистограмму в зависимости от выбранного континента.
5. Гистограмма должна отображать распределение `lifeExp` для выбранного континента.
6. Добавьте к гистограмме соответствующие заголовки и метки осей.

Решение

```
from dash import Dash, html, dcc, Input, Output, callback
import plotly.express as px

df = px.data.gapminder()

app = Dash(__name__)

app.layout = html.Div([
    dcc.Dropdown(
        id='continent-dropdown',
        options=[{'label': i, 'value': i} for i in df['continent'].unique()],
        value='Asia'
    ),
    dcc.Graph(id='life-exp-histogram')
])

@callback(
    Output('life-exp-histogram', 'figure'),
    Input('continent-dropdown', 'value')
)
def update_histogram(selected_continent):
    filtered_df = df[df['continent'] == selected_continent]
    fig = px.histogram(filtered_df, x='lifeExp', nbins=30, title=f'Life Expectancy Distribution in {selected_continent}')
    return fig

if __name__ == '__main__':
    app.run_server(debug=True)
```



ВОПРОСЫ



Заключение

